

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE CODE: ITA0607

COURSE NAME: MACHINE LEARNING

COURSE OUTCOME COVERED

CO4: K- Nearest Neighbour Learning – Locally weighted Regression – Radial Bases Functions – Case Based Learning **(BL 5)**

Assessment Tool Weightage: 40%

QUESTIONS: 03

Total Marks:25

Annexure A

Coding Challenge

Code of Conduct:

I VIMALA K 192424276 (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

<i>Criteria</i>	<i>Conceptual Understanding</i>	<i>Accuracy of Answers</i>	<i>Application of Knowledge</i>	<i>Completeness of Response</i>	<i>Clarity & Presentation</i>	<i>Total</i>
<i>Weightage</i>	30%	20%	15%	20%	15%	
<i>Q1</i>						
<i>Q2</i>						
<i>Q3</i>						

Signature of the student:

Total Marks :

Vimala K

Annexure A

Short Answer Test

1. Develop a Python program to implement a **Naïve Bayes classifier** for a given dataset. Compute prior and conditional probabilities, and classify a test instance. Display predicted class labels and evaluate performance using accuracy. Compare results with and without Laplace smoothing and analyze its impact on classification.

Coding:

```
import pandas as pd
```

```
data = {
    'Outlook': ['Sunny', 'Sunny', 'Overcast', 'Rain', 'Rain', 'Rain',
                'Overcast', 'Sunny', 'Sunny', 'Rain', 'Sunny', 'Overcast',
                'Overcast', 'Rain'],
    'Temperature': ['Hot', 'Hot', 'Hot', 'Mild', 'Cool', 'Cool',
                    'Cool', 'Mild', 'Cool', 'Mild', 'Mild', 'Mild',
                    'Hot', 'Mild'],
    'Humidity': ['High', 'High', 'High', 'High', 'Normal', 'Normal',
                 'Normal', 'High', 'Normal', 'Normal', 'Normal', 'High',
                 'Normal', 'High'],
    'Wind': ['Weak', 'Strong', 'Weak', 'Weak', 'Weak', 'Strong',
             'Strong', 'Weak', 'Weak', 'Weak', 'Strong', 'Strong',
             'Weak', 'Strong'],
    'Play': ['No', 'No', 'Yes', 'Yes', 'Yes', 'No',
             'Yes', 'No', 'Yes', 'Yes', 'Yes', 'Yes',
             'Yes', 'No']
}
```

```
df = pd.DataFrame(data)
```

```
features = ['Outlook', 'Temperature', 'Humidity', 'Wind']
target = 'Play'
```

```

def predict(data, test, smoothing=False):

    classes = data[target].unique()
    total = len(data)
    probabilities = {}

    for cls in classes:

        class_data = data[data[target] == cls]
        class_count = len(class_data)

        prior = class_count / total
        probability = prior

        print(f"\nClass: {cls}")
        print(f"Prior Probability = {prior:.4f}")

        for feature in features:

            value = test[feature]

            count = len(
                class_data[class_data[feature] == value]
            )

            if smoothing:
                k = data[feature].nunique()
                conditional = (count + 1) / (class_count + k)
            else:
                conditional = count / class_count

            print(
                f"P({feature}={value} | {cls}) = "
                f"{conditional:.4f}"
            )

            probability *= conditional

        probabilities[cls] = probability

    print(
        f"Final Probability = "
        f"{probability:.8f}"
    )

```

```

    )

    return max(probabilities, key=probabilities.get)

test_instance = {
    'Outlook': 'Sunny',
    'Temperature': 'Cool',
    'Humidity': 'High',
    'Wind': 'Strong'
}

print("=" * 50)
print("WITHOUT LAPLACE SMOOTHING")
print("=" * 50)

prediction_without = predict(
    df,
    test_instance,
    False
)

print("\nPredicted Class:",
      prediction_without)

print("\n" + "=" * 50)
print("WITH LAPLACE SMOOTHING")
print("=" * 50)

prediction_with = predict(
    df,
    test_instance,
    True
)

print("\nPredicted Class:",
      prediction_with)

def accuracy(data, smoothing):

```

```

correct = 0

for _, row in data.iterrows():

    test = {
        feature: row[feature]
        for feature in features
    }

    predicted = predict(
        data,
        test,
        smoothing
    )

    if predicted == row[target]:
        correct += 1

return correct / len(data) * 100

```

```

acc_without = accuracy(df, False)
acc_with = accuracy(df, True)

```

```

print("\n" + "=" * 50)
print("ACCURACY")
print("=" * 50)

```

```

print(
    "Accuracy without Laplace:",
    f"{acc_without:.2f}%"
)

```

```

print(
    "Accuracy with Laplace:",
    f"{acc_with:.2f}%"
)

```

```
Output

Prior Probability = 0.3571
P(Outlook=Rain|No) = 0.3750
P(Temperature=Mild|No) = 0.3750
P(Humidity=High|No) = 0.7143
P(Wind=Strong|No) = 0.5714
Final Probability = 0.02049927

Class: Yes
Prior Probability = 0.6429
P(Outlook=Rain|Yes) = 0.3333
P(Temperature=Mild|Yes) = 0.4167
P(Humidity=High|Yes) = 0.3636
P(Wind=Strong|Yes) = 0.3636
Final Probability = 0.01180638

=====
ACCURACY
=====
Accuracy without Laplace: 92.86%
Accuracy with Laplace: 92.86%

=== Code Execution Successful ===
```

2. Write a Python program to implement a simple **Genetic Algorithm** to optimize a mathematical function. Initialize a population, perform selection, crossover, and mutation operations, and evolve solutions over generations. Display fitness values and best solution, and analyze how parameters influence convergence speed and solution quality.

Coding:

```
import random
import math
```

```

def fitness(x):
    return x * math.sin(10 * math.pi * x) + 1

def initialize_population(size):
    return [
        random.uniform(0, 1)
        for _ in range(size)
    ]

def selection(population):
    candidates = random.sample(population, 3)
    return max(candidates, key=fitness)

def crossover(parent1, parent2):

    alpha = random.random()

    child1 = (
        alpha * parent1 +
        (1 - alpha) * parent2
    )

    child2 = (
        alpha * parent2 +
        (1 - alpha) * parent1
    )

    return child1, child2

def mutation(individual, mutation_rate):

    if random.random() < mutation_rate:

        individual += random.gauss(0, 0.1)

        individual = max(
            0,
            min(1, individual)

```

)

return individual

```
def genetic_algorithm(
    population_size=20,
    generations=30,
    crossover_rate=0.8,
    mutation_rate=0.1):

    population = initialize_population(
        population_size
    )

    best_solution = None
    best_fitness = float('-inf')

    print("GENETIC ALGORITHM")
    print("-" * 60)

    for generation in range(generations):

        current_best = max(
            population,
            key=fitness
        )

        current_fitness = fitness(
            current_best
        )

        if current_fitness > best_fitness:

            best_solution = current_best
            best_fitness = current_fitness

    print(
        f'Generation {generation + 1:2d}: '
        f'x = {current_best:.6f}, '
        f'Fitness = {current_fitness:.6f}'
    )
```

```

new_population = []

new_population.append(current_best)

while len(new_population) < population_size:

    parent1 = selection(population)
    parent2 = selection(population)

    if random.random() < crossover_rate:

        child1, child2 = crossover(
            parent1,
            parent2
        )

    else:

        child1 = parent1
        child2 = parent2

    child1 = mutation(
        child1,
        mutation_rate
    )

    child2 = mutation(
        child2,
        mutation_rate
    )

    new_population.append(child1)

    if len(new_population) < population_size:
        new_population.append(child2)

    population = new_population

print("-" * 60)

print(
    f"Best Solution = {best_solution:.6f}"

```

```
)

print(
    f"Best Fitness = {best_fitness:.6f}"
)

random.seed(42)

genetic_algorithm(
    population_size=20,
    generations=30,
    crossover_rate=0.8,
    mutation_rate=0.1
)
```

```
Output
Generation 12: x = 0.851190, Fitness = 1.850595
Generation 13: x = 0.851190, Fitness = 1.850595
Generation 14: x = 0.851190, Fitness = 1.850595
Generation 15: x = 0.851190, Fitness = 1.850595
Generation 16: x = 0.851190, Fitness = 1.850595
Generation 17: x = 0.851190, Fitness = 1.850595
Generation 18: x = 0.851190, Fitness = 1.850595
Generation 19: x = 0.851190, Fitness = 1.850595
Generation 20: x = 0.851190, Fitness = 1.850595
Generation 21: x = 0.851190, Fitness = 1.850595
Generation 22: x = 0.851190, Fitness = 1.850595
Generation 23: x = 0.851190, Fitness = 1.850595
Generation 24: x = 0.851190, Fitness = 1.850595
Generation 25: x = 0.851190, Fitness = 1.850595
Generation 26: x = 0.851190, Fitness = 1.850595
Generation 27: x = 0.851190, Fitness = 1.850595
Generation 28: x = 0.851190, Fitness = 1.850595
Generation 29: x = 0.851190, Fitness = 1.850595
Generation 30: x = 0.851190, Fitness = 1.850595
-----
Best Solution = 0.851190
Best Fitness = 1.850595
```

RUBRICS FOR CALCULATION

Criteria	Weightage	Level 4 – Exemplary (5 Marks)	Level 3 – Proficient (4 Marks)	Level 2 – Developing (2–3 Marks)	Level 1 – Beginning (0–1 Mark)
Conceptual Understanding	30	Demonstrates clear and complete understanding of the concept	Good understanding with minor gaps	Basic understanding with some misconceptions	Poor or incorrect understanding
Accuracy of Answer	20	Completely correct answer with no errors	Mostly correct with minor errors	Partially correct with noticeable errors	Incorrect or irrelevant answer
Application of Knowledge	15	Correctly applies concepts to solve the question/problem	Applies concepts with minor mistakes	Limited or partially correct application	No or incorrect application
Completeness of Response	20	Covers all required parts of the question	Covers most parts with minor omissions	Covers only some parts	Incomplete or missing response
Clarity & Presentation	15	Clear, concise, and well-organized answer	Understandable with minor issues	Somewhat unclear or poorly structured	Difficult to understand